

mmreal API Reference

This document describes every public function in `mmreal.h`. For an overview, build instructions, and integer-type configuration, see `README.md`.

Conventions

- **Index base.** All `(i)`, `(j)`, `(k)` indices are 0-based.
 - **Return on failure.** Pointer-returning functions return `NULL`; `bool`-returning functions return `false`; `MM_DBL`-returning functions return `NAN`; index-returning functions return `-1`. The cause is available via `mm_real_get_last_error`.
 - **Types.** `MM_INT` is the index/size type (default `long`); `MM_DBL` is the value type (default `double`). Both are defined in `mmio.h`.
 - **Symmetry.** Functions that require a *general* matrix reject symmetric input with `MM_ERROR_FORMAT_MISMATCH`.
-

Data types

`mm_real`, `mm_sparse`, `mm_dense`

Aliases for the same struct `s_mm_real`. The three names document intent; they are interchangeable at the type level.

```
c
struct s_mm_real {
    MM_typecode typecode; // Matrix Market typecode
    MMRealSymm  symm;     // symmetry property
    MM_INT      m, n, nnz; // rows, columns, non-zeros (m*n if dense)
    MM_INT      *i;       // CSC row indices (sparse only), size nnz
    MM_INT      *p;       // CSC column pointers (sparse only), size n+1
    MM_DBL      *data;    // values, size nnz
    bool        owner;    // true if this object owns and frees its arrays
};
```

`MMRealFormat`

`MM_REAL_SPARSE` (CSC) or `MM_REAL_DENSE` (column-major).

`MMRealSymm`

`MM_REAL_GENERAL`, `MM_REAL_SYMMETRIC_UPPER`, or `MM_REAL_SYMMETRIC_LOWER`.

`MMResult`

Error codes: `MM_SUCCESS`, `MM_ERROR_UNKNOWN`, `MM_ERROR_NOT_IMPLEMENTED`,
`MM_ERROR_NULL_ARGUMENT`, `MM_ERROR_INVALID_ARGUMENT`, `MM_ERROR_DIMENSION_MISMATCH`,
`MM_ERROR_INDEX_OUT_OF_BOUNDS`, `MM_ERROR_FORMAT_MISMATCH`, `MM_ERROR_ALLOCATION_FAILED`,
`MM_ERROR_FILE_IO`.

Type-checking macros

`mm_real_is_sparse(a)`, `mm_real_is_dense(a)`, `mm_real_is_symmetric(a)`, `mm_real_is_upper(a)`,
`mm_real_is_lower(a)`.

0. Support

`MMResult mm_real_get_last_error (void)`

Returns the last error code set on the current thread. The error state is thread-local. Call this after a function signals failure.

`const char *mm_result_to_string (MMResult res)`

Returns a constant, human-readable description of an `MMResult` code. Never returns `NULL`.

1. Creation, destruction, and copying

`mm_real *mm_real_new (MMRealFormat format, MMRealSymm symm, MM_INT m, MM_INT n, MM_INT nnz)`

Allocates a new matrix and zero-initializes its data array. For sparse matrices, `nnz` is the number of non-zeros to allocate; for dense, pass `nnz = m * n`. Dimensions must be positive, a symmetric matrix must be square, and each of `m`, `n`, `nnz` must fit in the BLAS integer range. Returns `NULL` on failure.

`void mm_real_free (mm_real *mm)`

Frees the object. The CSC arrays `i` and `p` are always freed; the `data` array is freed only if the object owns it (`owner == true`). Safe to call on `NULL`.

`mm_real *mm_real_copy (const mm_real *mm)`

Returns a newly allocated deep copy of any matrix (sparse or dense). The copy owns its data.

`mm_real *mm_real_eye (MMRealFormat type, MM_INT n)`

Returns a new `n x n` identity matrix in the requested format.

```
mm_real *mm_real_view_array (MMRealFormat format, MMRealSymm symm, MM_INT m, MM_INT n,
MM_INT nnz, MM_DBL *data)
```

Wraps an existing `(data)` array in an `(mm_real)` object **without copying or taking ownership** (`(owner == false)`). The caller retains responsibility for the array's lifetime. Destructive operations (in-place transpose, format conversion, reallocation) refuse to run on the resulting view. The same dimension/range checks as `(mm_real_new)` apply.

```
bool mm_real_realloc (mm_real *mm, MM_INT nnz)
```

Resizes the internal `(data)` (and, for sparse, `(i)`) arrays to hold `(nnz)` elements. Fails on a non-owning view. Returns `(true)` on success.

```
bool mm_real_resize (mm_real *x, MM_INT m, MM_INT n, MM_INT nnz, bool do_realloc)
```

Changes the recorded dimensions to `(m)`, `(n)`, `(nnz)`. If `(do_realloc)` is `(true)`, the arrays are reallocated to the new `(nnz)` first. Dimensions must fit in the BLAS integer range.

2. Element access and manipulation

```
MM_DBL mm_real_get (const mm_real *x, MM_INT i, MM_INT j)
```

Returns element `(i, j)`, or `(NAN)` on a bounds or null error. For symmetric matrices, indices are normalized to the stored triangle automatically. For sparse matrices, missing entries read as `(0.0)`.

```
bool mm_real_set (mm_real *x, MM_INT i, MM_INT j, MM_DBL val)
```

Sets element `(i, j)`. For sparse matrices, a new non-zero is inserted (with array reallocation and shifting) if the entry did not previously exist. Symmetric indices are normalized to the stored triangle.

```
bool mm_real_set_all (mm_real *mm, MM_DBL val)
```

Sets every stored element of the data array to `(val)`.

```
bool mm_real_memcpy (mm_real *dest, const mm_real *src)
```

Copies `(src)` into `(dest)`. The two must match in dimensions, symmetry, and format.

```
bool mm_real_transpose (mm_real *x)
```

Transposes in place. Dense square matrices are transposed directly; rectangular dense matrices use a temporary buffer. Sparse matrices use an `(O(m + n + nnz))` rebuild. Fails on a non-owning sparse view.

```
bool mm_real_sort (mm_real *x)
```

Sorts the non-zeros of a sparse matrix by row index within each column (required for correct `get`/binary-search behavior). A no-op for dense matrices.

3. Format and type conversion

```
bool mm_real_sparse_to_dense (mm_sparse *s)
```

Converts a sparse matrix to dense in place. Fails on a non-owning view.

```
bool mm_real_dense_to_sparse (mm_dense *d, MM_DBL threshold)
```

Converts a dense matrix to sparse in place, dropping elements with absolute value below `threshold`. Fails on a non-owning view.

```
bool mm_real_symmetric_to_general (mm_real *x)
```

Expands a symmetric matrix to an explicit general matrix in place. For sparse input, off-diagonal entries are mirrored into the other triangle; for dense input, the stored triangle is copied across the diagonal.

```
bool mm_real_general_to_symmetric (char uplo, mm_real *x)
```

Reduces a general matrix to symmetric in place, keeping the `'U'`/`'u'` (upper) or `'L'`/`'l'` (lower) triangle. For sparse input, entries in the discarded triangle are removed and arrays shrunk; for dense input, only metadata changes.

```
mm_dense *mm_real_copy_sparse_to_dense (const mm_sparse *s)
```

Returns a new dense copy of a sparse matrix (non-destructive).

```
mm_sparse *mm_real_copy_dense_to_sparse (const mm_dense *x, MM_DBL threshold)
```

Returns a new sparse copy of a dense matrix, dropping sub-`threshold` elements (non-destructive).

4. Matrix assembly and extraction

```
mm_real *mm_real_vertcat (const mm_real *x1, const mm_real *x2)
```

Returns `[x1; x2]`. Inputs must share format, be general, and have equal column counts.

```
mm_real *mm_real_horzcat (const mm_real *x1, const mm_real *x2)
```

Returns `[x1, x2]`. Inputs must share format, be general, and have equal row counts.

```
bool mm_real_xj_col_to (mm_real *xj, const mm_real *x, MM_INT j)
```

Writes column j of x into the pre-allocated dense vector xj (which must be dense with $xj \rightarrow m == x \rightarrow m$).

```
mm_dense *mm_real_xj_col (const mm_real *x, MM_INT j)
```

Returns column j as a newly allocated dense $m \times 1$ vector.

```
bool mm_real_xi_row_to (mm_real *xi, const mm_real *x, MM_INT i)
```

Writes row i of x into the pre-allocated dense vector xi (which must be dense with $xi \rightarrow m == x \rightarrow n$). Row extraction from sparse CSC input searches all columns and is comparatively slow.

```
mm_dense *mm_real_xi_row (const mm_real *x, MM_INT i)
```

Returns row i as a newly allocated dense $n \times 1$ vector.

5. Linear algebra: AXPY-like operations

```
bool mm_real_axpy (MM_DBL alpha, const mm_real *x, mm_real *y)
```

Computes $y = \alpha * x + y$ for all four sparse/dense combinations. x and y must match in dimensions and symmetry. Adding a dense x to a sparse y converts y to dense.

```
bool mm_real_axjpy (MM_DBL alpha, const mm_real *x, MM_INT j, mm_dense *y)
```

Computes $y = \alpha * x(:,j) + y$, where y is a dense general column vector with $y \rightarrow m == x \rightarrow m$ and $y \rightarrow n == 1$.

```
bool mm_real_scale (mm_real *x, MM_DBL alpha)
```

Scales every stored element: $x = \alpha * x$.

```
bool mm_real_xj_scale (mm_real *x, MM_INT j, MM_DBL alpha)
```

Scales column j by α . Requires a general matrix.

```
bool mm_real_add (mm_real *x, MM_DBL alpha)
```

Adds α to every element of a **dense** matrix. Rejected for sparse matrices (`MM_ERROR_NOT_IMPLEMENTED`), since it would densify them.

```
bool mm_real_xj_add (mm_real *x, MM_INT j, MM_DBL alpha)
```

Adds α to every element of column j of a **dense, general** matrix. Rejected for sparse matrices.

6. Linear algebra: products

```
MM_DBL mm_real_dot (const mm_real *x, const mm_real *y)
```

Returns the inner product $x' * y$ of two column vectors ($n == 1$) of equal length. Handles all sparse/dense combinations.

```
bool mm_real_x_dot_y (bool transx, bool transy, MM_DBL alpha, const mm_real *x, const mm_real *y, MM_DBL beta, mm_real *z)
```

Computes $z = \alpha * \text{op}(x) * \text{op}(y) + \beta * z$, where $\text{op}(M)$ is M or M' per the `trans` flags. z must be dense and general; y must be general. Inner and outer dimensions are validated. Dense-dense paths use `dgemm`/`dsymm`; other paths iterate column-by-column. Columns are processed in parallel via OpenMP where applicable.

```
bool mm_real_x_dot_yk (bool trans, MM_DBL alpha, const mm_real *x, const mm_real *y, MM_INT k, MM_DBL beta, mm_dense *z)
```

Computes one column: $z(:,k) = \alpha * \text{op}(x) * y(:,k) + \beta * z(:,k)$. y must be general, and k must be a valid column of both y and z . This is the matrix-times-single-column kernel underlying `mm_real_x_dot_y`.

```
bool mm_real_xj_trans_dot_y_to (mm_real *dest, const mm_real *x, MM_INT j, const mm_real *y)
```

Writes the row vector $\text{dest} = x(:,j)' * y$ into the pre-allocated `dest` (with $\text{dest->n} == y->n$). y must be general and share the row count of x .

```
mm_dense *mm_real_xj_trans_dot_y (const mm_real *x, MM_INT j, const mm_real *y)
```

Returns $x(:,j)' * y$ as a newly allocated dense $1 \times n$ row vector.

```
MM_DBL mm_real_xj_trans_dot_yk (const mm_real *x, MM_INT j, const mm_real *y, MM_INT k)
```

Returns the scalar $x(:,j)' * y(:,k)$. y must be general.

7. Vector / column statistics

All functions in this group operate on a single column j and return `NAN` (or `-1` for index functions) on error.

```
MM_INT mm_real_iamax (const mm_real *x)
```

Returns the index, within `x->data`, of the element with the largest absolute value. For sparse input this is the position in the non-zero array, not a matrix coordinate.

`MM_INT mm_real_xj_iamax (const mm_real *x, MM_INT j)`

Returns the local index of the largest-magnitude element in column `j` (row index for dense; position within the column's non-zeros for sparse).

`MM_DBL mm_real_xj_asum (const mm_real *x, MM_INT j)`

Sum of absolute values (L1 norm) of column `j`.

`MM_DBL mm_real_xj_sum (const mm_real *x, MM_INT j)`

Sum of the elements of column `j`.

`MM_DBL mm_real_xj_nrm2 (const mm_real *x, MM_INT j)`

Euclidean (L2) norm of column `j`.

`MM_DBL mm_real_xj_ssqr (const mm_real *x, MM_INT j)`

Sum of squares of column `j` (i.e. `nrm2` squared).

`MM_DBL mm_real_xj_mean (const mm_real *x, MM_INT j)`

Mean of column `j`, taken over all `m` rows (implicit zeros included for sparse).

`MM_DBL mm_real_xj_std (const mm_real *x, MM_INT j)`

Sample standard deviation of column `j` (divisor `m - 1`). Returns `0.0` when `m <= 1`.

8. File I/O

`mm_real *mm_real_fread (FILE *fp)`

Reads a matrix in Matrix Market format (coordinate→sparse, array→dense). Coordinate input may be unordered and may contain duplicate `(i, j)` entries; row indices are sorted within each column and duplicates are summed. Returns `NULL` on error.

`bool mm_real_fwrite (FILE *stream, const mm_real *x, const char *format)`

Writes `x` in Matrix Market format. `format` is a `printf` conversion for the value (e.g. `"%.16g"`). Indices are written 1-based per the format spec.

`bool mm_real_fprintf (FILE *stream, const mm_real *a, const char *format, char delim)`

Prints the matrix as a dense, human-readable grid with `delim` between columns, using `format` for each value.

`mm_real *mm_real_fread_binary (FILE *fp)`

Reads a matrix from the custom binary format. The 4-byte typecode tag selects sparse ("MCRS") or dense ("MARG"). The header is validated before allocation. The file must have been written with the same `MM_INT` width.

`bool mm_real_fwrite_binary (FILE *fp, const mm_real *x)`

Writes `x` in the custom binary format: a 4-byte typecode tag, then `m`, `n`, `nnz`, then the index arrays (sparse) and the data array.

Error-handling example

```
c
mm_real *A = mm_real_fread (fp);
if (A == NULL) {
    MMResult err = mm_real_get_last_error ();
    fprintf (stderr, "load failed: %s\n", mm_result_to_string (err));
    /* handle error */
}
```